

T12.3 - Servers

Application server

¿Por qué se pone un *proyecto en un servidor* y no en *la propia computadora*?

Porque se apuntan a ciertas *garantías*:

- Tener un contexto en el cual pueda correr el proyecto.
- La computadora debe estar encendida 24/7.
- La computadora debe estar conectada a internet 24/7.
- El sistema operativo no se debe actualizar. En caso de que sí lo haga, detiene todo.

Evolución del modelo Cliente/Servidor

1. Mainframe

El **mainframe** (servidor) hace referencia a la *supercomputadora central con mucha capacidad de cómputo* que ofrecía *servicio a muchos usuarios al mismo tiempo* de forma centralizada.

Los usuarios (clientes) accedían a través de *terminales "bobas"*, monitores que no procesaban nada por sí mismos, solo *enviaban lo que se le tipea* al mainframe y *mostraban lo que el mainframe responde*.

2. Aplicaciones de escritorio (PC)

El poder de cómputo pasó a la PC. Cada usuario tenía una. Ya no hay una supercomputadora que procesara todo por atrás.

Los *servidores quedaron con funciones restringidas*, ya sea de mail o espacios para intercambiar información.

La aplicación del usuario no corría más en el servidor. El cómputo de la aplicación corría 100% en la PC de uno.

En Excel, la aplicación vive en la computadora local. Puede funcionar sin conexión.

3. Navegadores con HTML estático

Todo el poder vuelve al servidor, en especial para páginas donde se lee información.

Para leer la información había que pedírsela a un servidor.

El navegador (cliente) hace un pedido, el servidor le responde con el HTML, el navegador lo muestra.

Esa dinámica pedido-respuesta entre dos roles bien diferenciados es el esquema Cliente/Servidor.

```
<h1>Bienvenidos</h1>
```

```
<p>Última actualización: marzo de 1996</p>
```

Bienvenidos

Última actualización: marzo de 1996

4. Navegadores con applets Java

El *servidor* sigue concentrando la *mayor parte del poder de cómputo*.

Los *navegadores* pueden *ejecutar aplicaciones descargadas* desde Internet.

Un **applet Java** es un programa que *se descarga desde el servidor y se ejecutan en el navegador*.

El *navegador ofrece un entorno de ejecución aparte* para que esas aplicaciones funcionen, en pos de agregar interactividad y lógica de negocio sin que haga falta instalar software adicional.

5. Navegadores con HTML dinámico

Los navegadores incorporan *motores de JavaScript capaces de ejecutar código* de manera local.

El *servidor* no solo entrega archivos, sino que *los genera dependiendo de quién pide qué* cosa.

Gmail no le muestra la misma página a todos. Arma una página específica para uno con sus correos.

Una parte de la lógica de la aplicación se ejecuta en el *cliente*, y *la otra parte en el servidor*.

En el cliente corre lo inmediato y visual: abrir un correo, marcarlo como leído y que cambie al instante...

En el servidor corre lo sensible y persistente: enviar el correo de verdad, autenticar tu sesión, almacenar tus mensajes...

Un correo nuevo combina ambas: el servidor lo recibe, lo guarda y lo filtra, y el cliente lo inserta en la lista en pantalla sin recargar la página.

Arquitecturas modernas

1. **Frontend** (React, Angular, Vue).
2. **Backend** (Java, Python, Node).
3. **Bases de datos.**
4. **APIsREST.**
5. **Servicios cloud.**

Actualidad

1. Grandes servidores.
2. Gran capacidad de cómputo en los clientes.
3. La información llega cruda de muchos lugares distintos.
4. El navegador hace todo el trabajo cuando entra a una página porque le llega esta información cruda. Debe procesarla y armar en la pantalla.